

DAY 5 · WORKED EXAMPLE

Worked Example

Guided LLM prompt chain to recreate a full machine-learning notebook

Example dataset: classification_workshop.csv

Task: defect prediction from process measurements

Participants see the complete path: prompts → notebook sections → code → metrics → model card → human audit.

Evidence → Prompt → Notebook → Results → Model card → Human validation

Worked example overview

What participants will produce by the end of the prompt-chain demonstration

INPUT FILE

classification_workshop.csv
220 process runs
Inputs: temperature, pressure, speed, vibration
Target: defect

LLM TASK

Generate notebook-ready markdown and code
Use exact column names
Include checks after each section

HUMAN ROLE

Approve target and unit
Audit leakage and metrics
Reject causal or deployment claims

Final product

- A runnable Jupyter notebook scaffold
- Prompt-response log for each step
- Baseline and logistic-regression comparison
- Confusion matrix and metric interpretation
- Initial model card with limits and next step

SUCCESS CRITERION

The notebook should not just run. You must be able to defend every modeling choice, every metric, and every conclusion.

Prompt-chain map

Seven short prompts replace one vague request



For each step, you collect three things:

- The exact prompt used
- The LLM-generated notebook section
- A human audit note: accept, revise, or reject

TEACHING POINT

Prompt chaining makes AI output inspectable. Each stage can be corrected before errors propagate into the final notebook.

Prompt 0 — Establish role and guardrails

Worked setup prompt used before generating any notebook code

COPY-PASTE PROMPT 0

You are helping create a Jupyter notebook for an applied AI workshop.

Use Python, pandas, matplotlib/seaborn, and scikit-learn.
Create short markdown explanations and executable code cells.

Do not invent dataset columns.
Do not claim causality from prediction results.
Do not remove outliers without a documented domain-based rule.
Preprocessing must be fitted only on training data.
Add one human-verification question after each section.

WORKED RESPONSE

The LLM acknowledges constraints and asks for dataset name and column list before writing code.

HUMAN AUDIT

Good response: asks for actual columns.
Bad response: immediately invents variables, deployment claims, or code using unknown column names.

CHECKPOINT

Do not continue until the dataset name and exact columns are provided.

Prompt 1 — Problem framing

Worked output for classification_workshop.csv

PROMPT 1

Dataset: classification_workshop.csv
Columns: temperature_c, pressure_kpa,
speed_mm_s, vibration_mm_s, defect

Create notebook-ready markdown and code that identify:
unit of observation, inputs, target,
model task, baseline, and metric.

HUMAN CHECK

Are all inputs available before final inspection? Is “defect”
defined consistently?

Worked framing table

Element	Worked example
Unit	One process run
Inputs	temperature, pressure, speed, vibration
Target	defect: 0 = acceptable, 1 = defective
Task	Binary classification
Baseline	Predict majority class
Metric	Recall/F1, not accuracy alone

APPROVED OUTPUT

Using process measurements, predict whether one run produces a defect;
compare to majority baseline and prioritize defect recall.

Prompt 2 — Data audit and visualization

Worked audit values become evidence for later prompts

PROMPT 2

Generate notebook cells to load the CSV and report:

- `df.head()`, `df.shape`, `df.dtypes`
- missing values and duplicates
- target class balance
- summary statistics
- count plot and process-variable plots

Worked audit output

Rows	Accept	Defect	Missing
220	176	44	0
process runs	80% class 0	20% class 1	in this file

EVIDENCE FROM AUDIT

Class imbalance is present. A majority baseline reaches 80% accuracy while detecting zero defects.

HUMAN CHECK

Did the LLM use actual columns? Did plots show class balance and feature ranges rather than decorative figures?

Prompt 3 — Leakage and availability review

Worked table: the LLM proposes, faculty verify

PROMPT 3

Prediction time: before final quality inspection.
Inputs: temperature_c, pressure_kpa,
speed_mm_s, vibration_mm_s.
Target: defect.

Create a table: variable, role,
available at prediction time?, leakage risk?, decision.

Variable	Available?	Risk	Decision
temperature_c	Yes	Low	Keep
pressure_kpa	Yes	Low	Keep
speed_mm_s	Yes	Low	Keep
vibration_mm_s	Yes	Low	Keep
defect	Outcome	Target	Do not use as input

HUMAN CHECK

The LLM cannot know timing unless you provide it. Add workflow context before accepting the keep/remove decisions.

APPROVED OUTPUT

No leakage detected among the four inputs, assuming all are recorded before final inspection.

Prompt 4 — Feature engineering with discipline checks

Worked example uses conservative feature suggestions

PROMPT 4

Suggest candidate features using temperature, pressure, speed, and vibration.

Separate into:

1. domain-informed
2. mathematically convenient
3. risky or possibly leaky

Provide code only for justified features.
Do not claim causality.

WORKED DECISION

For this simplified notebook, keep raw features only. Save engineered-feature ideas as next-step hypotheses.

Feature	Status	Rationale
temperature_deviation	Clarify	Need nominal temperature
pressure_deviation	Clarify	Need target pressure
vibration_speed_ratio	Test	May reflect instability per speed
future inspection flag	Reject	Outcome-derived leakage
raw timestamp	Clarify	May require shift/hour feature

TEACHING POINT

AI-generated features are hypotheses. They are not evidence until tested and validated.

Prompt 5 — Generate the reproducible pipeline

Worked code scaffold for the notebook

WORKED CODE CELL

```
X = df[features]
y = df["defect"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y,
    random_state=42
)

baseline = np.full(len(y_test), y_train.mode()[0])

model = Pipeline([
    ("scale", StandardScaler()),
    ("classifier", LogisticRegression(max_iter=1000))
])
model.fit(X_train, y_train)
pred = model.predict(X_test)
```

HUMAN AUDIT CHECKS

- ✓ Split before fitting model
- ✓ Baseline uses same test target
- ✓ StandardScaler is inside pipeline
- ✓ Fixed random_state used

WHAT TO REVISE IF GENERATED

Revise if the LLM fits scaling before train/test split, evaluates on training data, omits the baseline, or invents features.

NOTE

For categorical features, use ColumnTransformer and OneHotEncoder. This simplified file has numeric inputs only.

Worked model results

The output the notebook should reproduce

Baseline accuracy

0.800

predicts every run acceptable

Baseline recall

0.000

finds no defects

Model accuracy

0.855

slightly higher overall

Model precision

1.000

no false alarms

Model recall

0.273

misses most defects

Confusion matrix

	Pred accept	Pred defect
Actual accept	44	0
Actual defect	8	3

MODEL INTERPRETATION

The model is conservative: when it flags a defect it is correct, but it detects only 3 of 11 actual defects. It should not replace inspection.

Prompt 6 — Evaluation and error analysis

Worked interpretation separates evidence from claims

PROMPT 6

Interpret these results for a process engineer.
Separate:

- direct evidence
- reasonable interpretation
- unsupported conclusions
- most costly error
- recommended next analysis

Missed defects are more costly than false alarms.

HUMAN CHECK

If the LLM says the model is “ready for deployment,” revise.
The evidence supports limited screening value only.

Category	Worked response
Direct evidence	Accuracy rose from 0.800 to 0.855; recall is 0.273
Interpretation	Model is cautious and misses many defects
Unsupported	Process variables cause defects
Costly error	False negatives: 8 missed defects
Next analysis	Threshold sweep, class weighting, cross-validation

Prompt 7 — Completed model card

Worked final documentation section

PROMPT 7

Create an initial model card with:

- intended use
- out-of-scope use
- dataset and target
- baseline result
- model result
- primary error pattern
- supported conclusion
- unsupported conclusion
- limitations
- recommended next analysis

DELIVERABLE

One concise evidence-grounded paragraph plus the structured model-card table.

Field	Worked entry
Intended use	Teaching example; inspection prioritization prototype
Out-of-scope	Automatic part approval or production deployment
Result	Acc 0.855, precision 1.000, recall 0.273
Error pattern	Missed 8 of 11 actual defects
Supported	Inputs contain some predictive signal
Do not claim	Variables cause defects
Next step	Tune threshold and collect more defect examples